CIFAR-10 Classification with a Regularized Residual Network

Zihan Xu | Kaggle team: **Zihan Xu**
NYU Tandon Deep Learning, Fall 2026 | November 7, 2026

Public code, weights and executed notebook:
https://github.com/Zihan-Xu/DL-project1

1. Overview and main findings

This project develops a residual convolutional network for the course CIFAR-10 classification task under a five-million-parameter limit. The submitted model combines a widened ResNet-20 with anti-aliased downsampling and several regularizers. After a complete three-seed confirmation study, the selected recipe was trained from scratch on all 50,000 permitted labeled images. Inference uses one checkpoint and averages predictions from the original image and its horizontal mirror.

Final submitted model	Recorded result
Kaggle public accuracy	87.90% (0.87900), actual second submission
Trainable parameters	2,436,346, including the classification head
Final fit	50,000 images; 240 epochs; seed 42; final-epoch weights
Research validation	96.04% mean; 0.151 percentage-point SD across three seeds
Independent local / private accuracy	No independent local holdout after full-data fitting; Kaggle private score not recorded

The second submission improved the recorded public score from 85.00% to 87.90%, a gain of 2.90 percentage points. The 96.04% result belongs to earlier models trained on 40,000 images and evaluated on a shared 5,000-image validation set; it is not the test accuracy of the final full-data model.

2. Architecture and design rationale

Component	Structure / output
Input and stem	32 x 32 RGB; 3 x 3 convolution, 48 channels, BN, ReLU
Residual stage 1	Three two-convolution blocks; 48 channels; 32 x 32
Residual stage 2	Three blocks; 96 channels; BlurPool transition to 16 x 16
Residual stage 3	Three blocks; 192 channels; BlurPool transition to 8 x 8
Classifier	Global average pooling; linear 192-to-10 head

Residual shortcuts support optimization [1]. The 3x channel width adds capacity while using 48.73% of the parameter budget. Fixed 3 x 3 binomial BlurPool filters precede stride-two sampling on both residual and projection paths [2]. They add compute but no trainable weights. One-view inference requires approximately 436.1 million multiply-accumulate operations; two-view inference doubles that cost.

3. Data protocol and training methodology

The authorized dataset contains 50,000 labeled 32 x 32 RGB images, balanced across ten classes, plus 2,000 competition images with hidden labels. Development used a fixed class-stratified split of 40,000 training, 5,000 validation and 5,000 historical local-test images. Each development model used normalization statistics from its training portion. The competition labels were not available for training or local validation.

Research models were selected by validation accuracy, with ties resolved in favor of the earlier epoch. After configuration and inference policy were frozen, the deployment model was reinitialized with seed 42 and fitted to all 50,000 labeled examples. This deliberately absorbs the former validation and local-test sets into training. The final checkpoint is epoch 240, without early stopping or a new independent local accuracy estimate.

Setting	Exact final training configuration
Optimizer and loss	SGD; initial learning rate 0.1; momentum 0.9; weight decay 0.0005; cross-entropy
Schedule and batch	Cosine decay over 240 epochs, no warmup; batch size 128; final batch size 80
Sampling	Each image once per epoch; shuffled order repaired by swaps so every batch contains all ten classes
Horizontal flip	Probability 0.5, applied after channel normalization
Rotation	Probability 0.5; uniform angle within +/-10 degrees; bilinear sampling with reflection padding
Cutout	Probability 0.5; 12 x 12 mask clipped to image boundaries; fill with normalized zero [3]
Dropout	Probability 0.1 before the second convolution in every residual block [4]
Residual dropping	Per-sample probability increases linearly from 0.1/9 to 0.1 across nine blocks; retained paths scaled by inverse keep probability [5]
Numerical execution	BF16 CUDA training; FP32 inference; deterministic algorithms; TF32 disabled

Full-data RGB means are [0.49139968, 0.48215841, 0.44653091], and standard deviations are [0.24703223, 0.24348513, 0.26158784]. They are computed solely from the permitted training images. The schedule produces 391 optimizer updates per epoch, 93,840 updates in total and 12 million training-image presentations.

Dropout and residual dropping discourage reliance on particular feature paths. Their cost is a noisier optimization problem, motivating a controlled comparison of the 80- and 240-epoch budgets. Residual dropping is implemented as masking after convolution, so it does not save convolutional compute. The cosine schedule follows the decay shape associated with SGDR [6], without warm restarts.

The submitted recipe uses no random cropping, Mixup, external images, pretrained weights, teacher model or attention module. During inference, all training regularizers are disabled. Horizontal-flip test-time augmentation (TTA) averages the two softmax probability vectors with equal weights, then selects the largest probability. Rotation is a training augmentation only.

4. Confirmation experiments and selection

An earlier combination study supplied the common base: 3x ResNet-20, BlurPool, Cutout12 and weight decay 0.0005. The confirmation study tested a full 2 x 2 x 2 factorial combination of rotation (R), dropout (D) and residual dropping (S) at 80 epochs, plus base-only and all-factor configurations at 240 epochs. All ten configurations were trained with seeds 142, 143 and 144: 30 completed runs.

For each run, raw and TTA predictions were computed at the same checkpoint selected by raw validation accuracy. All comparisons below use the same 5,000 validation examples and three seeds. Selection maximized the total number correct across seeds among the 20 configuration/policy choices; ties favored raw inference, fewer epochs and then configuration name.

R / D / S	Epochs	Raw mean %	TTA mean %
000	80	94.827	95.233
001	80	95.047	95.387
010	80	95.080	95.240
011	80	94.927	95.267
100	80	95.173	95.527
101	80	95.313	95.533
110	80	95.233	95.467
111	80	95.440	95.660
000	240	95.427	95.733
111	240	95.967	96.040

Table 1. A bit of 1 activates the corresponding factor: R = +/-10-degree rotation, D = dropout 0.1, S = maximum residual-drop probability 0.1. Every row has the same parameter count, 2,436,346. The last row is the selected long_111 recipe.

Selected recipe seed	Raw accuracy %	TTA accuracy %
142	95.92	96.06
143	96.02	95.88
144	95.96	96.18

At matched budgets, the all-factor recipe improves raw mean accuracy over the base by 0.613 percentage points at 80 epochs and 0.540 points at 240 epochs. Extending the all-factor recipe from 80 to 240 epochs adds 0.527 points. These observations support the selected configuration but do not isolate a universally reliable effect for each regularizer.

For long_111, TTA adds 0.073 points on average; its unadjusted paired 95% t interval is [-0.396, 0.543] points. Two seeds improve and one declines. None of 34 planned comparisons passes joint Holm correction at 0.05. Three seeds quantify training randomness on one split, not performance on three independent datasets. Repeated validation-based design and checkpoint selection can make the reported validation mean optimistic.

5. Results, learning curves and lessons

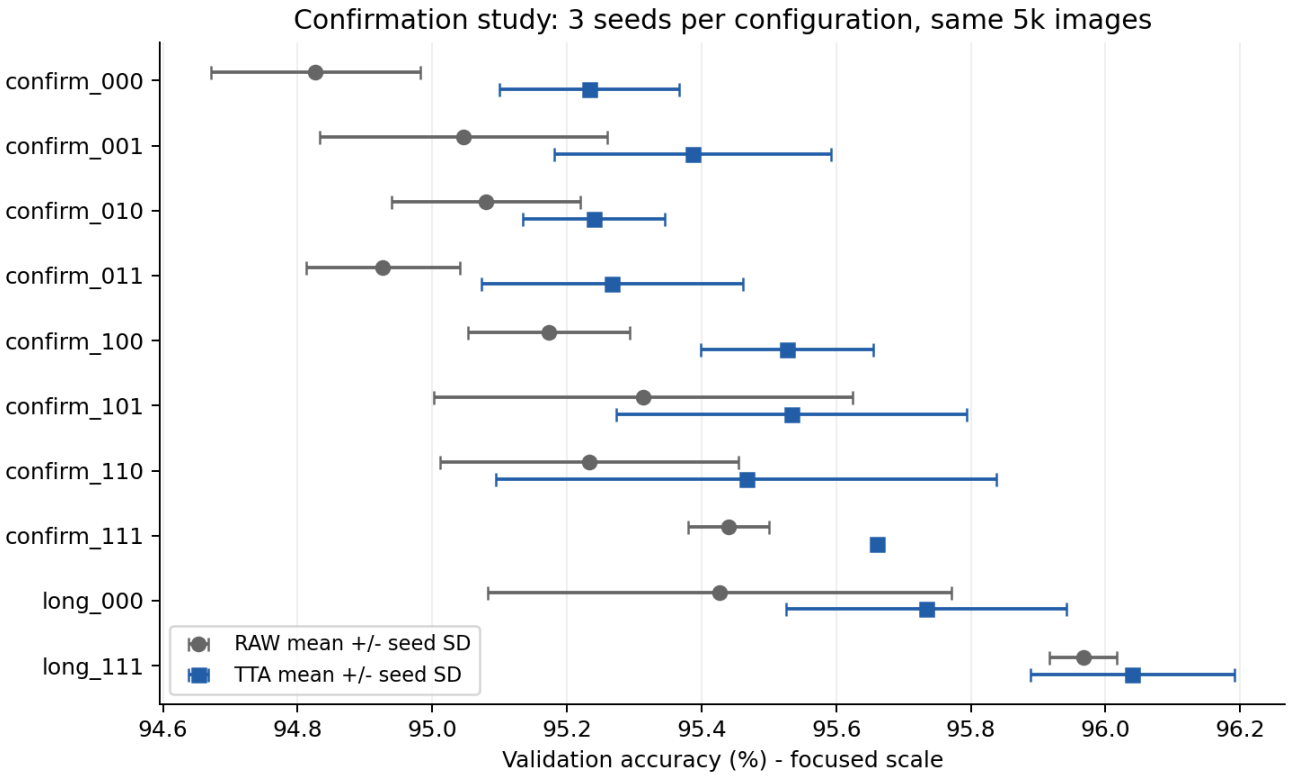


Figure 1. Confirmation validation accuracy. Error bars show the standard deviation across three seeds, not a confidence interval. The horizontal axis uses a focused accuracy scale. Both inference policies use the same checkpoint for each run.

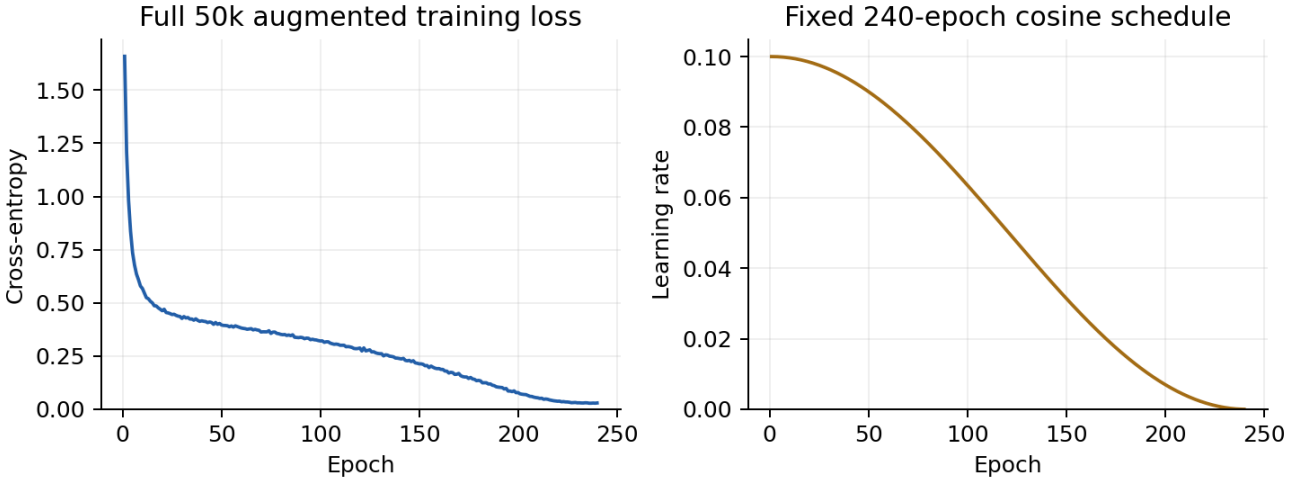


Figure 2. The final 50,000-image fit: augmented training loss and learning rate across all 240 epochs. Training loss describes optimization; it is not an independent generalization measurement.

Submission	Public accuracy	Interpretation
Earlier core240, raw	85.00%	Earlier 40k-trained model; user-confirmed submission
Second long_111, TTA	87.90%	Final 50k-trained model; downloaded CSV hash verified

The second submission was recorded at 22:15:04 on November 6, 2026 (America/New_York). Its downloaded CSV exactly matches the 2,000 predictions generated by the released checkpoint. The 2.90-point public improvement reflects a package of changes: regularization, full-data fitting and TTA. It cannot be attributed to one component from this leaderboard comparison.

6. Reproducibility and limitations

The public repository opens on the standalone submitted-version branch. It includes the original final checkpoint, exact configuration, model and augmentation code, training and evaluation entry points, complete logs, per-image confirmation predictions, two valid inference policies and an executed review notebook. The notebook explicitly prints 87.90% public accuracy and 2,436,346 trainable parameters. Earlier experiments remain available in the archive branch.

To reproduce the submitted labels, obtain the course data and run `python predict.py --data-dir data/competition --output reproduced_tta.csv --compare submissions/01_RECOMMENDED_full50k_tta/submission.csv`. The command reads the original test PNGs, applies the stored normalization and fixed TTA policy, checks sample order and compares every predicted label with the submitted file.

The environment used for training was Python 3.13.5, PyTorch 2.11.0+cu128, CUDA 12.8, NumPy 2.1.3 and Pillow 11.1.0 on one NVIDIA RTX 5070 Ti. CPU prediction is supported. Floating-point behavior can differ across hardware and library versions; versions and checksums are recorded. Training can be resumed with optimizer, scheduler and random-number states.

The cleaned implementation preserves checkpoint tensor names and was checked against the original initialization, inference, training forward/backward computation and augmentation. Its first full training epoch reproduced every non-timing log field exactly. Independent CPU replay reproduced both the 2,000 TTA labels and the 2,000 raw labels. The original complete 240-epoch fit is preserved; it was not repeated during code cleanup.

The main limitations are a reused validation split, only three confirmation seeds, uncertain individual effects after multiple-comparison correction and no independent local holdout for the final full-data model. The private Kaggle score is not recorded. Future work should use a fresh validation design before expanding architecture or augmentation searches. The submitted model provides a reproducible 87.90% public result while remaining well below the course parameter limit.

References

[1] K. He, X. Zhang, S. Ren, and J. Sun. Deep Residual Learning for Image Recognition. CVPR, 2016.
https://arxiv.org/abs/1512.03385

[2] R. Zhang. Making Convolutional Networks Shift-Invariant Again. ICML, 2019.
https://arxiv.org/abs/1904.11486

[3] T. DeVries and G. W. Taylor. Improved Regularization of Convolutional Neural Networks with Cutout. 2017.
https://arxiv.org/abs/1708.04552

[4] N. Srivastava et al. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR 15, 2014, pp. 1929-1958.
https://jmlr.org/papers/v15/srivastava14a.html

[5] G. Huang et al. Deep Networks with Stochastic Depth. ECCV, 2016.
https://arxiv.org/abs/1603.09382

[6] I. Loshchilov and F. Hutter. SGDR: Stochastic Gradient Descent with Warm Restarts. ICLR, 2017. Cosine decay is used here without restarts.
https://arxiv.org/abs/1608.03983

[7] NYU Tandon. Deep Learning Project 1, Fall 2026. Course assignment and Kaggle competition rules.
https://www.kaggle.com/competitions/cifar-10-res-net-architecture-challenge-dl/rules

Implementation: local project code inspired by the cited methods. All training uses permitted course data and random initialization.